

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

Error Analysis Task

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0405

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 30%

Duration: 30 minutes

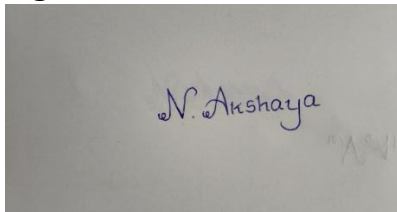
Marks: 25

Code of Conduct:

I **N.AKSHAYA(192425129)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in blue ink. The signature reads "N. Akshaya".

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

- Identify the conceptual error in the likelihood formulation.
- Explain how this mistake affects parameter estimation.
- Suggest the correct mathematical formulation and reasoning.
- How would using log-likelihood fix computational issues?

Answer:

a) Identify the conceptual error in the likelihood formulation.

The student incorrectly defines the likelihood function as the **sum of probabilities**:

$$L(\theta) = \sum_{i=1}^n P(x_i|\theta)$$

For independent observations, the correct likelihood is the **product of probabilities**:

$$L(\theta) = \prod_{i=1}^n P(x_i|\theta)$$

The likelihood represents the **joint probability** of observing the complete dataset given the model parameters.

b) Explain how this mistake affects parameter estimation.

Using the sum instead of the product results in:

- Incorrect likelihood function.
- Wrong parameter estimation.
- Optimization converges to incorrect values.
- Poor classification accuracy.
- Increasing the dataset size cannot correct this mathematical error.

c) Suggest the correct mathematical formulation and reasoning.

Correct likelihood:

$$L(\theta) = \prod_{i=1}^n P(x_i|\theta)$$

Log-likelihood:

$$\log L(\theta) = \sum_{i=1}^n \log P(x_i|\theta)$$

The logarithm preserves the maximum while simplifying optimization.

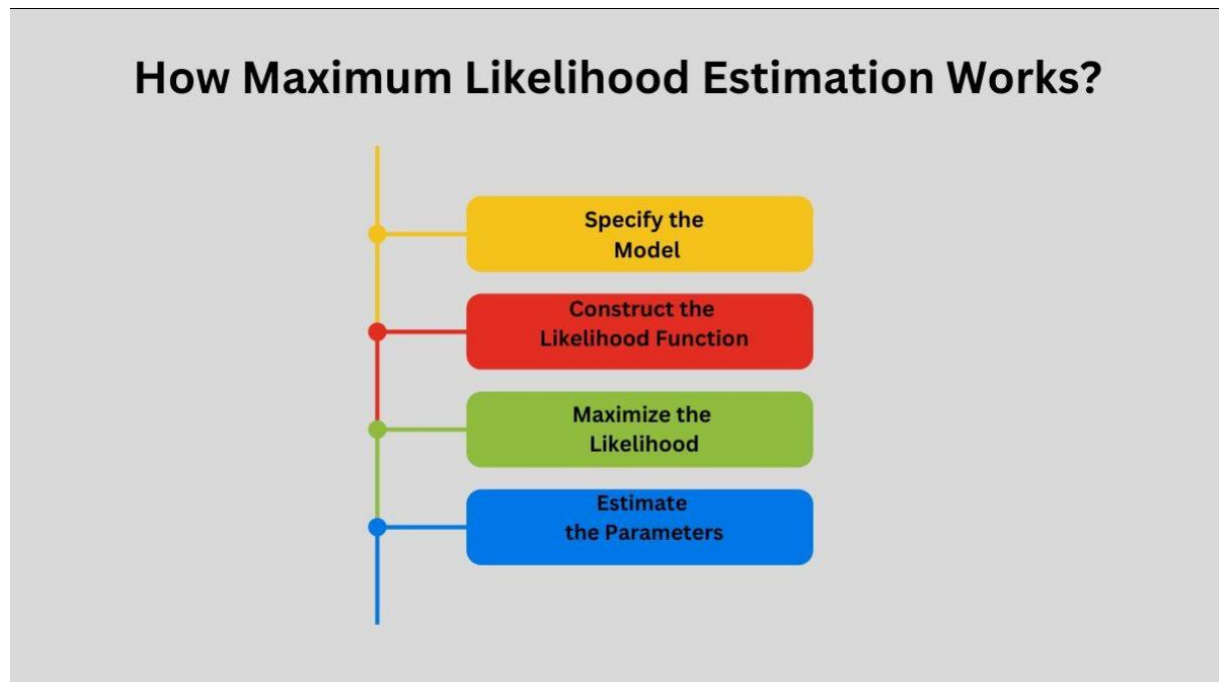
d) How would using log-likelihood fix computational issues?

- Prevents numerical underflow.

- Converts multiplication into addition.
- Simplifies differentiation.
- Provides stable optimization.
- Improves computational efficiency.

Conclusion:

The analysis shows that the likelihood function must be formulated as the **product of probabilities** rather than their sum to correctly estimate model parameters. Using the log-likelihood function improves numerical stability, simplifies optimization, and enables efficient gradient-based learning. Therefore, the correct mathematical formulation of MLE is essential for obtaining accurate parameter estimates and achieving high classification performance.



2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

- List possible implementation errors in the learning algorithm.
- Analyze how incorrect learning rate or weight updates affect convergence.
- Identify dataset-related issues that may cause this problem.
- Propose modifications to ensure convergence.

Answer:

a) Possible implementation errors

- Incorrect weight update equation
- Wrong bias update
- Incorrect learning rate
- Wrong activation function
- Incorrect label encoding
- Programming errors

- Improper stopping condition

b) Effect of incorrect learning rate

Weight update equation:

$$w = w + \eta(y - \hat{y})x$$

If the learning rate is:

- Too small $\rightarrow$ Very slow convergence.
- Too large $\rightarrow$ Oscillation and instability.
- Incorrect weight updates $\rightarrow$ Wrong decision boundary.

c) Dataset-related issues

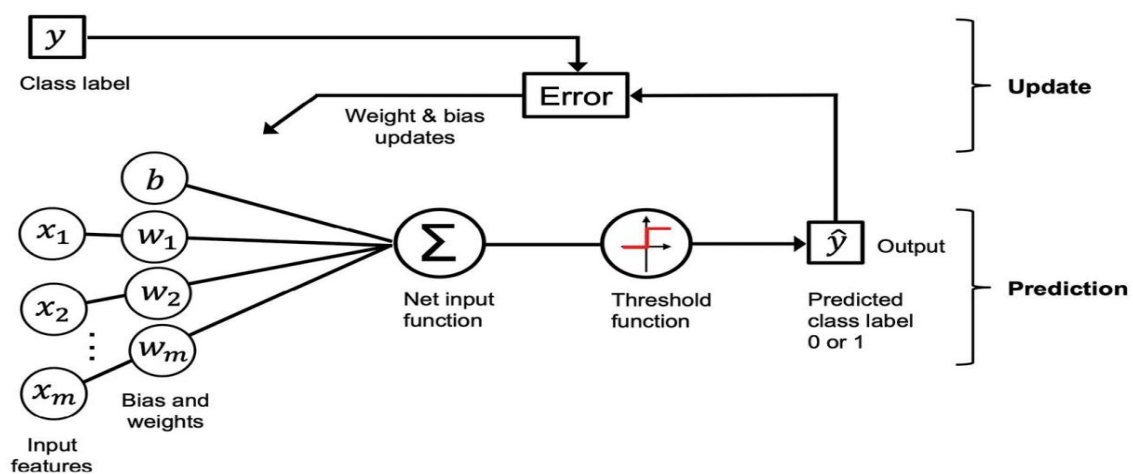
- Incorrect class labels
- Missing values
- Feature scaling problems
- Noisy samples
- Duplicate records
- Dataset not linearly separable

d) Proposed modifications

- Normalize input data.
- Verify weight update implementation.
- Use appropriate learning rate.
- Shuffle data after every epoch.
- Remove noisy samples.
- Verify linear separability.

Conclusion

The Perceptron algorithm converges successfully only when implementation errors, learning rate selection, and dataset preprocessing are handled correctly. Proper weight updates, normalized features, accurate labels, and suitable learning parameters ensure efficient convergence and reliable classification on linearly separable datasets.



3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

- a) Identify possible mathematical or coding errors in gradient computation.
- b) Explain the role of activation functions in gradient flow.
- c) Analyze how vanishing or exploding gradients affect training.
- d) Suggest techniques to fix the issue (e.g., normalization, initialization).

Answer:

a) Mathematical or coding errors

Possible errors include:

- Incorrect chain rule
- Wrong derivative calculation
- Incorrect learning rate
- Wrong loss function
- Programming bugs
- Improper weight initialization

b) Role of activation functions

Activation functions introduce non-linearity.

Examples:

- Sigmoid
- Tanh
- ReLU

ReLU reduces the vanishing gradient problem, whereas Sigmoid and Tanh may produce very small gradients.

c) Vanishing and exploding gradients

Vanishing Gradient

- Very small gradients
- Slow learning
- Early layers stop updating

Exploding Gradient

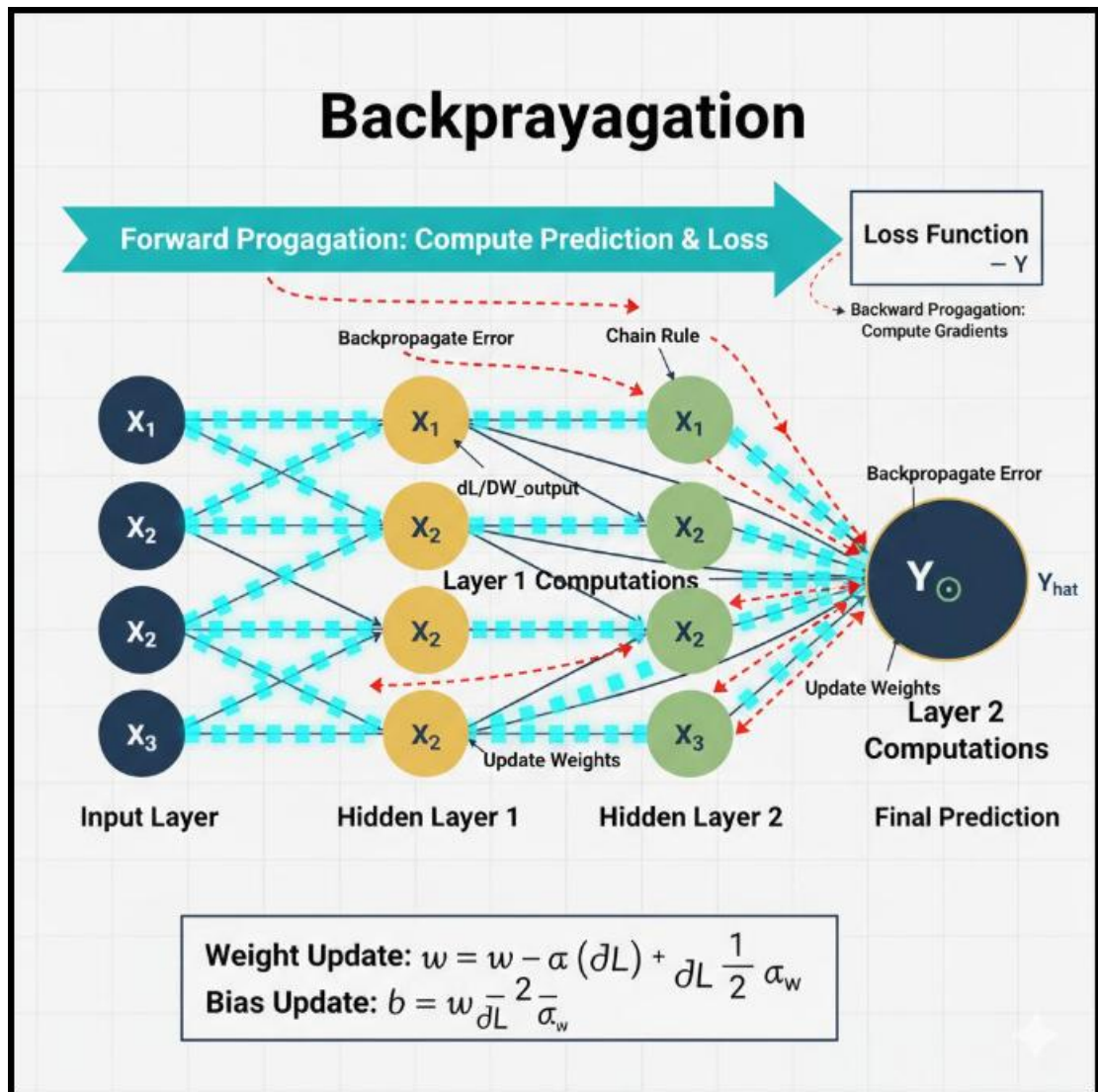
- Extremely large gradients
- Large weight updates
- Unstable training
- Divergence of loss

d) Techniques to fix the issue

- Xavier Initialization
- He Initialization
- Batch Normalization
- Gradient Clipping
- ReLU activation
- Adam optimizer
- Residual Networks (ResNet)

Conclusion

Successful backpropagation depends on correct gradient computation, appropriate activation functions, and proper weight initialization. Problems such as vanishing and exploding gradients can significantly reduce learning performance. Techniques including ReLU activation, Xavier or He initialization, Batch Normalization, Gradient Clipping, and adaptive optimizers improve convergence and enable deep neural networks to learn effectively.



4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

- Explain why batch gradient descent is slow in large datasets.
- Analyze the cause of fluctuations in SGD.
- Compare the error behavior of Mini-Batch Gradient Descent.
- Recommend the best approach and justify with error trade-offs.

Answer:

a) Why Batch Gradient Descent is slow

Batch Gradient Descent computes gradients using the entire dataset.

Therefore,

- High computational cost
- Large memory requirement
- Slow parameter updates
- Slow convergence on large datasets

b) Cause of SGD fluctuations

SGD updates parameters after every training sample.

Hence,

- High variance gradients
- Noisy updates
- Oscillating loss function
- Faster but less stable optimization

c) Error behavior of Mini-Batch Gradient Descent

Mini-Batch Gradient Descent combines Batch GD and SGD.

Advantages:

- Faster convergence
- Stable updates
- Lower computational cost
- Better generalization

d) Recommended approach

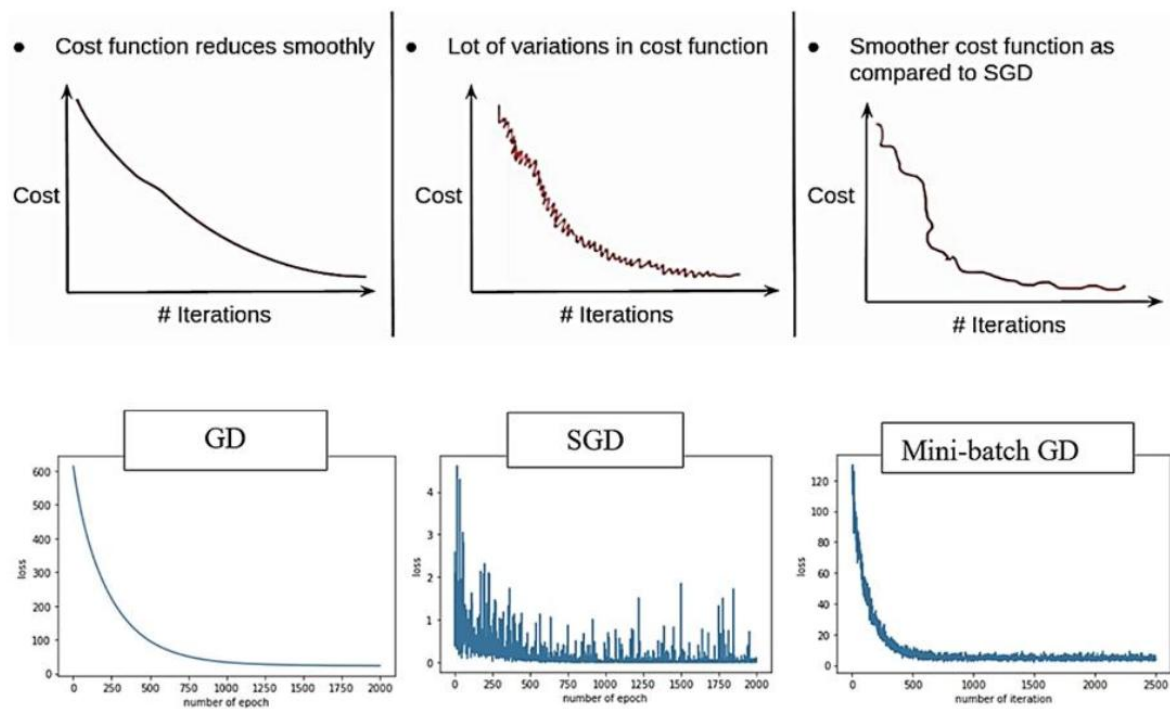
Mini-Batch Gradient Descent is recommended because it balances:

- Computational efficiency
- Stable convergence
- Lower variance
- Better prediction accuracy

Method	Speed	Stability
Batch GD	Slow	High
SGD	Fast	Low
Mini-Batch GD	Fast	High

Conclusion

Batch Gradient Descent provides stable optimization but is computationally expensive for large datasets, whereas Stochastic Gradient Descent offers faster updates with higher variance. Mini-Batch Gradient Descent combines the advantages of both approaches by achieving faster convergence, improved stability, and better computational efficiency, making it the preferred optimization method for most deep learning applications.



5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

- Explain how high dimensionality increases model error.
- Identify signs of overfitting in this scenario.
- Analyze why distance-based learning fails in high dimensions.
- Suggest dimensionality reduction or regularization techniques to improve performance.

Answer:

a) How high dimensionality increases error

As the number of features increases,

- Data becomes sparse.
- Model complexity increases.
- Overfitting becomes more likely.
- Generalization decreases.
- More training data is required.

b) Signs of overfitting

- High training accuracy
- Low testing accuracy
- Large training-testing accuracy gap
- Poor prediction on unseen data

c) Why distance-based learning fails

Distance-based algorithms depend on meaningful distances.

In high-dimensional spaces,

- Distances become nearly identical.
- Nearest neighbors lose significance.

- Similarity measures become ineffective.
- Classification performance decreases.

d) Dimensionality reduction and regularization techniques

Dimensionality Reduction

- Principal Component Analysis (PCA)
- Linear Discriminant Analysis (LDA)
- Autoencoders

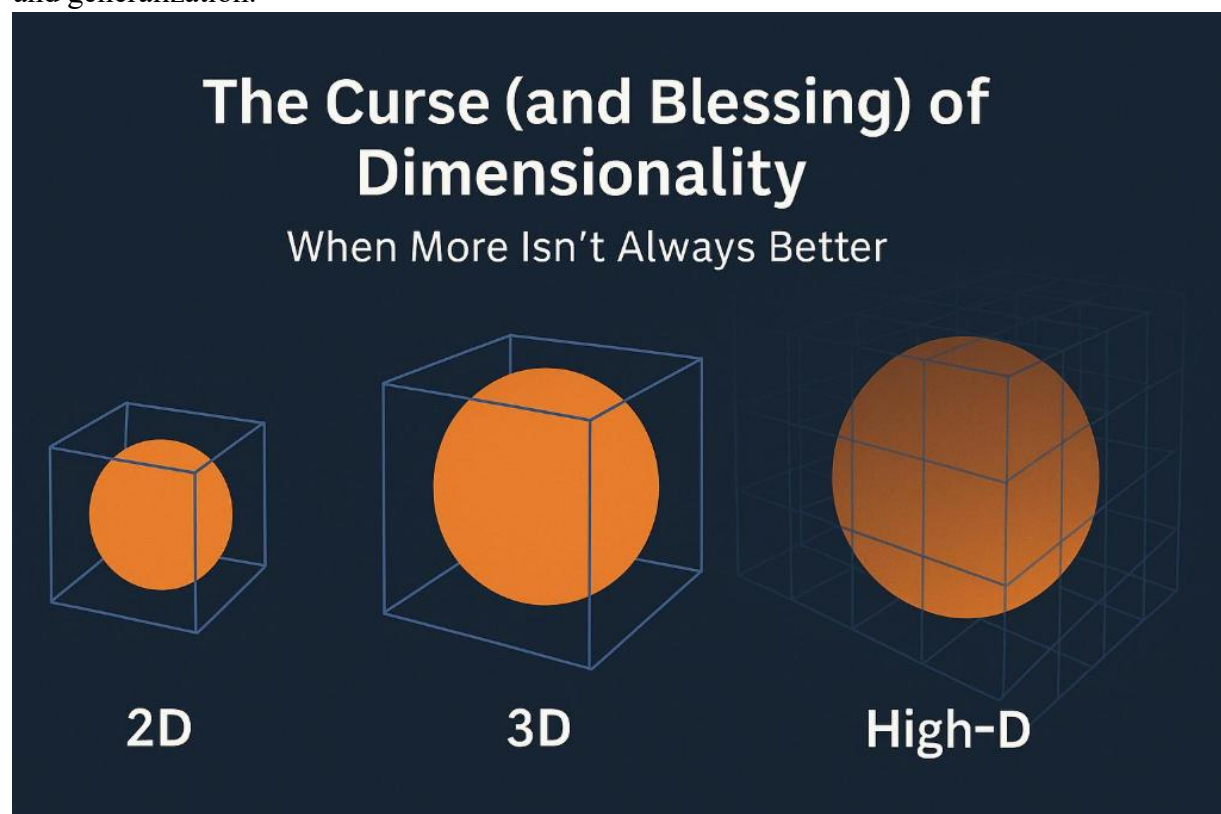
Regularization

- L1 Regularization
- L2 Regularization
- Dropout
- Early Stopping
- Feature Selection

These techniques reduce overfitting and improve model generalization.

Conclusion

High-dimensional data increases model complexity and often causes overfitting and poor generalization. Distance-based algorithms become less effective because distances between data points lose discriminatory power. Applying dimensionality reduction techniques such as PCA and LDA, together with regularization methods like L1/L2 regularization, Dropout, and feature selection, reduces model complexity and significantly improves prediction accuracy and generalization.



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand